

Let's make an instance representing a specific dog:

```
class Dog:
    --snip--
```

```
❶ my_dog = Dog('Willie', 6)

❷ print(f"My dog's name is {my_dog.name}.")
❸ print(f"My dog is {my_dog.age} years old.")
```

The Dog class we're using here is the one we just wrote in the previous example. Here, we tell Python to create a dog whose name is 'Willie' and whose age is 6 ❶. When Python reads this line, it calls the `__init__()` method in Dog with the arguments 'Willie' and 6. The `__init__()` method creates an instance representing this particular dog and sets the name and age attributes using the values we provided. Python then returns an instance representing this dog. We assign that instance to the variable `my_dog`. The naming convention is helpful here; we can usually assume that a capitalized name like Dog refers to a class, and a lowercase name like `my_dog` refers to a single instance created from a class.

Accessing Attributes

To access the attributes of an instance, you use dot notation. We access the value of `my_dog`'s attribute name ❷ by writing:

```
my_dog.name
```

Dot notation is used often in Python. This syntax demonstrates how Python finds an attribute's value. Here, Python looks at the instance `my_dog` and then finds the attribute name associated with `my_dog`. This is the same attribute referred to as `self.name` in the class Dog. We use the same approach to work with the attribute age ❸.

The output is a summary of what we know about `my_dog`:

```
My dog's name is Willie.
My dog is 6 years old.
```

Calling Methods

After we create an instance from the class Dog, we can use dot notation to call any method defined in Dog. Let's make our dog sit and roll over:

```
class Dog:
    --snip--

my_dog = Dog('Willie', 6)
my_dog.sit()
my_dog.roll_over()
```
